MINI PROJECT REPORT

AI23521 -BUILD AND DEPLOYMENT OF MACHINE LEARNING APPLICATIONS

STUDENT PERFORMANCE PREDICTION SYSTEM



Submitted by

SARADHA.S

241501187

Department of Artificial Intelligence and Machine Learning

RAJALAKSHMI ENGINEERING COLLEGE

Academic Year: 2024-2028

CONTENTS

S. No.	Section
1	Abstract
2	Introduction
3	Problem Statement and Objectives
4	Scope
5	Tools and Technologies
6	Dataset Description
7	System Architecture
8	Methodology and Algorithms
9	Model Evaluation
10	Flask API and Docker
11	Results and Discussion
12	Advantages and Limitations
13	Future Enhancements
14	Conclusion
15	References
Appendix	Execution Instructions

1. ABSTRACT

The Student Performance Prediction System demonstrates an integrated machine learning workflow using student-related features such as daily study hours, attendance, assignment marks, previous examination marks, and sleep hours. The system classifies performance into Low, Average, and High categories. It includes data preprocessing, binary and multiclass classification, Support Vector Machine (SVM), Random Forest, Decision Tree, K-Means clustering, Principal Component Analysis (PCA), feedforward and convolutional neural networks, a Generative Adversarial Network (GAN), Grid Search with cross-validation, and model deployment through a Flask REST API. Docker configuration files are also generated. The supplied dataset is synthetic and intended for educational demonstration; results should not be interpreted as validated predictions about real students.

2. INTRODUCTION

Machine learning can be used to explore patterns in structured educational data. Study habits, attendance, assignment performance, and previous marks are examples of features that can be analyzed to demonstrate classification and clustering methods. This project brings the topics from the BDML experiments into one workflow, from data preparation through model evaluation and deployment.

The project compares classical machine learning algorithms with neural-network approaches. It also introduces an unsupervised learning workflow and demonstrates how a trained model can be exposed through a REST API. The purpose is academic learning and algorithm practice, not automated grading or high-impact decisions.

3. PROBLEM STATEMENT AND OBJECTIVES

Problem statement: Develop a demonstration system that uses student-related numerical features to predict a performance category, compares multiple algorithms, explores data patterns, tunes model parameters, and provides predictions through an API.

- Generate or load a structured student-performance dataset.
- Preprocess numerical features and create training and testing subsets.
- Perform binary and multiclass classification with SVM and Random Forest.
- Train and visualize a Decision Tree classifier.
- Apply K-Means clustering and PCA.
- Train feedforward and one-dimensional convolutional neural networks.
- Demonstrate GAN-based synthetic feature generation.
- Use Grid Search and cross-validation to tune an SVM model.
- Save the selected model and expose predictions using Flask.
- Generate Docker configuration files for containerization.

4. SCOPE OF THE PROJECT

The project is designed for coursework and can be run in Google Colab using a CSV or Excel dataset. It demonstrates supervised learning, unsupervised learning, deep learning, model tuning, visualization, and deployment. The provided data is simulated; a real deployment would require representative data, consent and privacy safeguards, careful validation, fairness testing, and human oversight.

5. TOOLS AND TECHNOLOGIES

Tool / Library	Purpose
Python	Main programming language
Google Colab	Cloud notebook environment
Pandas and NumPy	Data manipulation and numerical processing
Matplotlib	Charts and visualizations
Scikit-learn	Preprocessing, classifiers, clustering, PCA, evaluation, and tuning
TensorFlow / Keras	FNN, CNN, and GAN demonstration
Joblib	Saving and loading the trained model
Flask	Building the REST API
Requests	Testing API endpoints
Docker	Containerizing the application in a Docker-enabled environment

6. DATASET DESCRIPTION

The included dataset contains 1,000 synthetic records and seven columns. Five numerical columns are used as prediction features. FinalScore is used to derive the target label and is not supplied to the prediction model, avoiding direct target leakage.

Column	Description	Role
StudyHours	Daily study time in hours	Input
Attendance	Attendance percentage	Input
AssignmentMarks	Assignment score (0–100)	Input
PreviousMarks	Previous examination score (0–100)	Input
SleepHours	Daily sleep duration in hours	Input
FinalScore	Generated score bounded between 0 and 100	Used to create label
Performance	Low, Average, or High	Target

Performance labels are generated from FinalScore: scores below 50 are Low, scores from 50 to below 75 are Average, and scores of 75 or higher are High. The records are generated using a formula with random variation. Therefore, model scores reflect this simulated process rather than real-world student outcomes.

7. SYSTEM ARCHITECTURE

1. Data source: load the student-performance CSV or Excel file.
2. Preprocessing: select the five input features and split records into training and testing sets.
3. Classification: train SVM, Random Forest, and Decision Tree models.
4. Unsupervised learning: apply K-Means to standardized features and use PCA for two-dimensional visualization.
5. Deep learning: train an FNN and CNN, and demonstrate GAN-based synthetic feature generation.
6. Evaluation: compare test metrics and tune SVM parameters using five-fold cross-validation.
7. Deployment: save the tuned model, create a Flask API, test a local prediction request, and generate Docker files.

8. METHODOLOGY AND ALGORITHMS

8.1 Data preprocessing

The data is loaded using Pandas. The five input features are separated from the Performance target. An 80:20 training-test split is created with stratification and a fixed random seed. StandardScaler is used where scaling is required, such as SVM, clustering, and neural-network training.

8.2 Binary and multiclass classification

For binary classification, High performance is encoded as 1 and Low or Average as 0. SVM and Random Forest are trained and tested on this binary target. For multiclass classification, the original Low, Average, and High labels are retained. SVM, Random Forest, and Decision Tree models are compared.

8.3 Decision Tree

A Decision Tree learns a series of feature-based rules to classify records. A limited tree depth is used to make the plotted decision rules easier to inspect. Its performance is evaluated with accuracy, a classification report, and a confusion matrix.

8.4 Random Forest and feature importance

Random Forest combines multiple decision trees to produce a classification. Its feature-importance scores provide a way to inspect which input features contributed most to the fitted forest. These scores describe the trained model and do not establish that a feature causes better performance.

8.5 K-Means and PCA

K-Means groups standardized feature vectors into three clusters. PCA projects the standardized data into two principal components for visualization. Clusters are discovered without using the target labels and do not necessarily correspond to Low, Average, and High performance.

8.6 Feedforward neural network and CNN

The feedforward neural network contains dense layers with ReLU activation and a softmax output for three classes. The CNN applies one-dimensional convolution and pooling to the ordered five-feature vector. Both models are trained with the Adam optimizer and categorical cross-entropy. The CNN is included for syllabus coverage and is not assumed to be the best choice for tabular data.

8.7 GAN

The GAN contains a generator that maps random latent vectors to scaled feature records and a discriminator that learns to distinguish original from generated records. Alternating training steps demonstrate the adversarial process. With a small synthetic dataset and short training, generated values may not reproduce the data distribution reliably.

8.8 Grid Search and cross-validation

GridSearchCV evaluates combinations of the SVM regularization parameter C, kernel, and gamma using five-fold cross-validation on the training set. The selected estimator is then evaluated on the held-out test set. This keeps final test evaluation separate from parameter selection.

9. MODEL EVALUATION

Accuracy measures the proportion of correct predictions. Precision measures the proportion of predicted members of a class that are correct; recall measures how many actual members of a class were identified; and F1-score combines precision and recall. A confusion matrix shows the distribution of correct and incorrect predictions across classes. The notebook prints the actual scores when executed.

Model	Metric to record	Actual value
Binary SVM	Test accuracy	Fill from Colab output
Binary Random Forest	Test accuracy	Fill from Colab output
Multiclass SVM	Test accuracy	Fill from Colab output
Multiclass Random Forest	Test accuracy	Fill from Colab output
Decision Tree	Test accuracy	Fill from Colab output
Feedforward neural network	Test accuracy	Fill from Colab output
CNN	Test accuracy	Fill from Colab output
Tuned SVM	Cross-validation and test accuracy	Fill from Colab output

Actual metric values should be copied from the executed notebook; they should not be estimated or invented. Because labels are derived from a formula, high scores on this dataset do not establish real-world predictive validity.

10. FLASK REST API AND DOCKER

10.1 REST API endpoints

HTTP method	Endpoint	Purpose
GET	/	Returns a message indicating that the API is running.
POST	/predict	Accepts five numerical features and returns the predicted performance category.

10.2 Example request

```
POST /predict
Content-Type: application/json

{
  "features": [5.0, 85.0, 78.0, 72.0, 7.0]
}
```

Example response structure (the predicted category depends on the fitted model):

```
{
  "predicted_performance": "Average"
}
```

10.3 Docker configuration

The project generates app.py, requirements.txt, and a Dockerfile. The Dockerfile installs dependencies, copies the saved model and application, exposes port 5000, and starts the Flask server. Run the following commands from a directory containing all required files on a system with Docker installed:

```
docker build -t student-performance .
docker run -p 5000:5000 student-performance
```

Colab can generate the files and test the local API, but a Docker daemon is generally not available in a standard Colab runtime.

11. RESULTS AND DISCUSSION

The notebook is designed to display the dataset preview, class distribution, study-hours scatter plot, model accuracy comparison, Decision Tree visualization, confusion matrices, Random Forest feature importance, K-Means/PCA plot, FNN and CNN training curves, GAN-generated samples, tuned SVM parameters, and a Flask JSON prediction response.

The results illustrate the different roles of the algorithms: Decision Trees provide visible rules, Random Forest combines trees, SVM uses a margin-based decision boundary, K-Means identifies unlabeled groups, and PCA supports visualization. The best SVM settings are selected by cross-validation. Insert screenshots from the actual Colab output into this section when preparing the final submitted copy.

CODE:

```
!pip install flask scikit-learn joblib requests pandas matplotlib openpyxl tensorflow -q


import os, time, socket, threading

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import joblib

import requests


from google.colab import files

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import StandardScaler, MinMaxScaler

from sklearn.pipeline import make_pipeline

from sklearn.svm import SVC

from sklearn.ensemble import RandomForestClassifier

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.cluster import KMeans
```

```
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay, classification_report
from flask import Flask, request, jsonify
from werkzeug.serving import make_server

print("Upload student_performance_dataset.csv or .xlsx")
uploaded = files.upload()
filename = next(iter(uploaded))

if filename.lower().endswith(".csv"):
    df = pd.read_csv(filename)
else:
    df = pd.read_excel(filename)

features = [
    "StudyHours", "Attendance", "AssignmentMarks",
    "PreviousMarks", "SleepHours"
]

X = df[features]
y = df["Performance"]

class_names = ["Low", "Average", "High"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nEXPERIMENT 1: PREPROCESSING")
```

```
print("Dataset Shape:", df.shape)
print(df.head())
print("\nMissing Values:\n", df.isnull().sum())
print("\nClass Distribution:\n", y.value_counts())

plt.figure(figsize=(7, 4))
y.value_counts().reindex(class_names).plot(kind="bar")
plt.title("Student Performance Distribution")
plt.xlabel("Performance")
plt.ylabel("Number of Students")
plt.show()

plt.figure(figsize=(7, 4))
plt.scatter(df["StudyHours"], df["FinalScore"], alpha=0.5)
plt.xlabel("Study Hours")
plt.ylabel("Final Score")
plt.title("Study Hours vs Final Score")
plt.show()

print("\nEXPERIMENT 2: BINARY AND MULTICLASS CLASSIFICATION")

yb = (y == "High").astype(int)
Xb_train, Xb_test, yb_train, yb_test = train_test_split(
    X, yb, test_size=0.2, random_state=42, stratify=yb
)

binary_svm = make_pipeline(StandardScaler(), SVC())
binary_svm.fit(Xb_train, yb_train)
```

```
binary_rf = RandomForestClassifier(n_estimators=100, random_state=42)
binary_rf.fit(Xb_train, yb_train)

print("Binary SVM Accuracy:",
      round(accuracy_score(yb_test, binary_svm.predict(Xb_test))*100, 2), "%")
print("Binary Random Forest Accuracy:",
      round(accuracy_score(yb_test, binary_rf.predict(Xb_test))*100, 2), "%")

print("\nEXPERIMENT 3: DECISION TREE")

dt_model = DecisionTreeClassifier(max_depth=5, random_state=42)
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)

print("Decision Tree Accuracy:",
      round(accuracy_score(y_test, dt_pred)*100, 2), "%")
print(classification_report(y_test, dt_pred, zero_division=0))

ConfusionMatrixDisplay(
    confusion_matrix(y_test, dt_pred, labels=class_names),
    display_labels=class_names
).plot(cmap="Blues")
plt.title("Decision Tree Confusion Matrix")
plt.show()

plt.figure(figsize=(16, 7))
plot_tree(dt_model, feature_names=features,
          class_names=dt_model.classes_, filled=True,
          rounded=True, max_depth=3)
```

```
plt.title("Decision Tree Visualization")
plt.show()

print("\nEXPERIMENT 4A AND 4B: SVM AND RANDOM FOREST")

svm_model = make_pipeline(StandardScaler(), SVC())
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)

models = {
    "SVM": svm_model,
    "Random Forest": rf_model,
    "Decision Tree": dt_model
}

scores = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    scores[name] = accuracy_score(y_test, pred)
    print(name, "Accuracy:", round(scores[name]*100, 2), "%")

plt.figure(figsize=(7, 4))
plt.bar(scores.keys(), [v*100 for v in scores.values()])
plt.ylabel("Accuracy (%)")
plt.title("Model Accuracy Comparison")
plt.ylim(0, 105)
plt.show()
```

```
plt.figure(figsize=(8, 4))
plt.barh(features, rf_model.feature_importances_)
plt.title("Random Forest Feature Importance")
plt.show()

print("\nEXPERIMENT 5: K-MEANS AND PCA")

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(7, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap="viridis")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clustering with PCA")
plt.show()

print("Cluster Counts:", np.bincount(clusters))
print("PCA Variance Explained:",
    round(sum(pca.explained_variance_ratio_)*100, 2), "%")

print("\nEXPERIMENT 6: FEEDFORWARD AND CONVOLUTIONAL NEURAL NETWORKS")
```

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Dense, Conv1D, Flatten, MaxPooling1D, Dropout
from tensorflow.keras.utils import to_categorical

tf.random.set_seed(42)

label_map = {"Low": 0, "Average": 1, "High": 2}
ytr_num = y_train.map(label_map).to_numpy()
yte_num = y_test.map(label_map).to_numpy()

nn_scaler = StandardScaler()
Xtr = nn_scaler.fit_transform(X_train)
Xte = nn_scaler.transform(X_test)

ytr = to_categorical(ytr_num, 3)
yte = to_categorical(yte_num, 3)

fnn = Sequential([
    Input(shape=(len(features),)),
    Dense(32, activation="relu"),
    Dense(16, activation="relu"),
    Dense(3, activation="softmax")
])

fnn.compile(optimizer="adam",
            loss="categorical_crossentropy", metrics=["accuracy"])

fnn_history = fnn.fit(
```

```
    Xtr, ytr, epochs=15, batch_size=32,
    validation_split=0.2, verbose=0
)

_, fnn_acc = fnn.evaluate(Xte, yte, verbose=0)
print("FNN Accuracy:", round(fnn_acc*100, 2), "%")

cnn = Sequential([
    Input(shape=(len(features), 1)),
    Conv1D(16, 2, activation="relu", padding="same"),
    MaxPooling1D(2),
    Flatten(),
    Dense(16, activation="relu"),
    Dropout(0.2),
    Dense(3, activation="softmax")
])

cnn.compile(optimizer="adam",
            loss="categorical_crossentropy", metrics=["accuracy"])

cnn_history = cnn.fit(
    Xtr[..., np.newaxis], ytr, epochs=15,
    batch_size=32, validation_split=0.2, verbose=0
)

_, cnn_acc = cnn.evaluate(Xte[..., np.newaxis], yte, verbose=0)
print("CNN Accuracy:", round(cnn_acc*100, 2), "%")

plt.figure(figsize=(7, 4))
```

```
plt.plot(fnn_history.history["accuracy"], label="Training")
plt.plot(fnn_history.history["val_accuracy"], label="Validation")
plt.title("FNN Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

plt.figure(figsize=(7, 4))
plt.plot(cnn_history.history["accuracy"], label="Training")
plt.plot(cnn_history.history["val_accuracy"], label="Validation")
plt.title("CNN Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

print("\nEXPERIMENT 7: GENERATIVE ADVERSARIAL NETWORK")

from tensorflow.keras.layers import LeakyReLU

gan_scaler = MinMaxScaler()
X_gan = gan_scaler.fit_transform(X).astype("float32")
latent_dim = 8
nf = len(features)

generator = Sequential([
    Input(shape=(latent_dim,)),
    Dense(32, activation="relu"),
```

```
    Dense(64, activation="relu"),
    Dense(nf, activation="sigmoid")
])

discriminator = Sequential([
    Input(shape=(nf,)),
    Dense(32),
    LeakyReLU(negative_slope=0.2),
    Dense(16, activation="relu"),
    Dense(1, activation="sigmoid")
])

discriminator.compile(optimizer="adam", loss="binary_crossentropy")
discriminator.trainable = False

gan = Sequential([generator, discriminator])
gan.compile(optimizer="adam", loss="binary_crossentropy")
discriminator.trainable = True

for epoch in range(100):
    idx = np.random.randint(0, len(X_gan), 32)
    real = X_gan[idx]
    noise = np.random.normal(0, 1, (32, latent_dim))
    fake = generator.predict(noise, verbose=0)

    discriminator.train_on_batch(real, np.ones((32, 1)))
    discriminator.train_on_batch(fake, np.zeros((32, 1)))

    noise = np.random.normal(0, 1, (32, latent_dim))
```

```
    discriminator.trainable = False
    gan.train_on_batch(noise, np.ones((32, 1)))
    discriminator.trainable = True

synthetic_scaled = generator.predict(
    np.random.normal(0, 1, (5, latent_dim)), verbose=0
)
synthetic_data = gan_scaler.inverse_transform(synthetic_scaled)

print("Synthetic Student Records:")
print(pd.DataFrame(synthetic_data, columns=features).round(2))

plt.figure(figsize=(7, 4))
plt.scatter(X_gan[:, 0], X_gan[:, 1], label="Original", alpha=0.4)
plt.scatter(synthetic_scaled[:, 0], synthetic_scaled[:, 1],
            label="Synthetic")
plt.xlabel("Scaled Study Hours")
plt.ylabel("Scaled Attendance")
plt.title("GAN Synthetic Data Demonstration")
plt.legend()
plt.show()

print("\nEXPERIMENT 8: GRID SEARCH AND CROSS-VALIDATION")

grid = GridSearchCV(
    make_pipeline(StandardScaler(), SVC()),
    {
        "svc__C": [0.1, 1, 10],
        "svc__kernel": ["linear", "rbf"],
```

```
        "svc__gamma": ["scale", "auto"]
    },
    cv=5, scoring="accuracy", n_jobs=-1
)

grid.fit(X_train, y_train)
best_model = grid.best_estimator_
best_pred = best_model.predict(X_test)

print("Best Parameters:", grid.best_params_)
print("Cross-Validation Accuracy:",
      round(grid.best_score_*100, 2), "%")
print("Test Accuracy:",
      round(accuracy_score(y_test, best_pred)*100, 2), "%")

ConfusionMatrixDisplay(
    confusion_matrix(y_test, best_pred, labels=class_names),
    display_labels=class_names
).plot(cmap="Blues")
plt.title("Tuned SVM Confusion Matrix")
plt.show()

joblib.dump(best_model, "student_model.pkl")

print("\nEXPERIMENT 9: FLASK REST API AND DOCKER")

app = Flask(__name__)

@app.route("/")
```

```
def home():
    return jsonify({"message": "Student Performance API is running"})

@app.route("/predict", methods=["POST"])
def predict():
    try:
        values = request.get_json()["features"]
        if len(values) != 5:
            return jsonify({"error": "Enter exactly 5 feature values"}), 400
        result = str(best_model.predict([values])[0])
        return jsonify({"predicted_performance": result})
    except Exception as e:
        return jsonify({"error": str(e)}), 400

sock = socket.socket()
sock.bind(("127.0.0.1", 0))
port = sock.getsockname()[1]
sock.close()

server = make_server("127.0.0.1", port, app)
thread = threading.Thread(target=server.serve_forever, daemon=True)
thread.start()
time.sleep(1)

url = f"http://127.0.0.1:{port}"
sample = X_test.iloc[0].tolist()

print("API Home:", requests.get(url).json())
response = requests.post(url + "/predict", json={"features": sample})
```

```
print("Sample Features:", sample)
print("Prediction:", response.json())
print("HTTP Status:", response.status_code)

with open("app.py", "w") as f:
    f.write('''from flask import Flask, request, jsonify
import joblib

app = Flask(__name__)
model = joblib.load("student_model.pkl")

@app.route("/")
def home():
    return jsonify({"message": "Student Performance API is running"})

@app.route("/predict", methods=["POST"])
def predict():
    try:
        values = request.get_json()["features"]
        if len(values) != 5:
            return jsonify({"error": "Enter exactly 5 feature values"}), 400
        return jsonify({"predicted_performance": str(model.predict([values])[0])})
    except Exception as e:
        return jsonify({"error": str(e)}), 400

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
''')
```

```
with open("requirements.txt", "w") as f:
    f.write("flask\nscikit-learn\njoblib\nnumpy\n")

with open("Dockerfile", "w") as f:
    f.write("""FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY student_model.pkl .
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
""")

df.to_csv("student_performance_output.csv", index=False)

print("\nCreated Files:")
for f in ["student_performance_output.csv", "student_model.pkl",
          "app.py", "requirements.txt", "Dockerfile"]:
    print(f, os.path.exists(f))

print("\nDocker commands for a Docker-enabled computer:")
print("docker build -t student-performance .")
print("docker run -p 5000:5000 student-performance")

server.shutdown()

print("Download the output files from the Colab Files panel.")
```

12. ADVANTAGES AND LIMITATIONS

12.1 Advantages

- Combines the nine BDML experiment topics into one workflow.
- Runs in Google Colab and accepts a CSV or Excel dataset.
- Produces multiple charts and evaluation outputs.
- Demonstrates classical machine learning and deep learning.
- Shows model saving, API testing, and Docker file generation.

12.2 Limitations

- The supplied dataset is synthetic, not collected from actual students.
- High accuracy may reflect the formula used to generate target labels.
- CNN and GAN are included for learning objectives and may not be ideal for small tabular data.
- GAN-generated records may be unrealistic.
- The API is tested locally and is not publicly hosted by default.
- Docker execution requires a Docker-enabled environment.

13. FUTURE ENHANCEMENTS

- Use a representative, ethically collected, anonymized student dataset.
- Test the model on data from a different semester or institution.
- Add API input validation and a user-friendly web interface.
- Evaluate calibration, fairness, and performance across relevant groups.
- Build a dashboard for visualizing model outputs and feature trends.
- Deploy to a suitable server only after validation and privacy review.
- Keep predictions advisory and avoid using them as the sole basis for consequential student decisions.

14. CONCLUSION

The Student Performance Prediction System demonstrates an integrated workflow covering data preparation, classification, clustering, dimensionality reduction, neural networks, GAN-based synthetic-data generation, model tuning, and REST API deployment. The tuned SVM is saved and served through Flask, and Docker configuration files are generated for containerization. The project provides practical experience with the BDML syllabus. Since the supplied dataset is synthetic, future work should validate the methods on appropriate real-world data before drawing conclusions about student performance.

15. REFERENCES

- Scikit-learn documentation — https://scikit-learn.org/stable/
- Pandas documentation — https://pandas.pydata.org/docs/
- Matplotlib documentation — https://matplotlib.org/stable/
- TensorFlow documentation — https://www.tensorflow.org/guide
- Flask documentation — https://flask.palletsprojects.com/
- Docker documentation — https://docs.docker.com/

APPENDIX: EXECUTION INSTRUCTIONS

8. Download student_performance_dataset.csv or student_performance_dataset.xlsx.
9. Open Google Colab and create a new Python notebook.
10. Paste the combined project code into one cell and run it.
11. Upload the dataset when the notebook prompts you.
12. Wait for training, visualizations, API testing, and file creation to finish.
13. Record actual scores and take screenshots of the generated outputs.
14. Download the model, app.py, requirements.txt, and Dockerfile from the Colab Files panel if needed.
15. Run Docker commands only in an environment where Docker is installed.

Prediction API input order:

```
1. StudyHours
2. Attendance
3. AssignmentMarks
4. PreviousMarks
5. SleepHours
```